FILMS DATABASE USING NEO4J

TEAM MEMBERS:
23MIS7242-MEENAKSHI.KOLLA
23MIS7275-RATNA KUMARI.DAGGUBATI

COURSE NAME:NOSQL DATABASE

COURSE CODE:CSE-3009

SLOT: L39+L40

FACULTY:DR.KARTHIKA NATARAJAN

1.Creating a Node

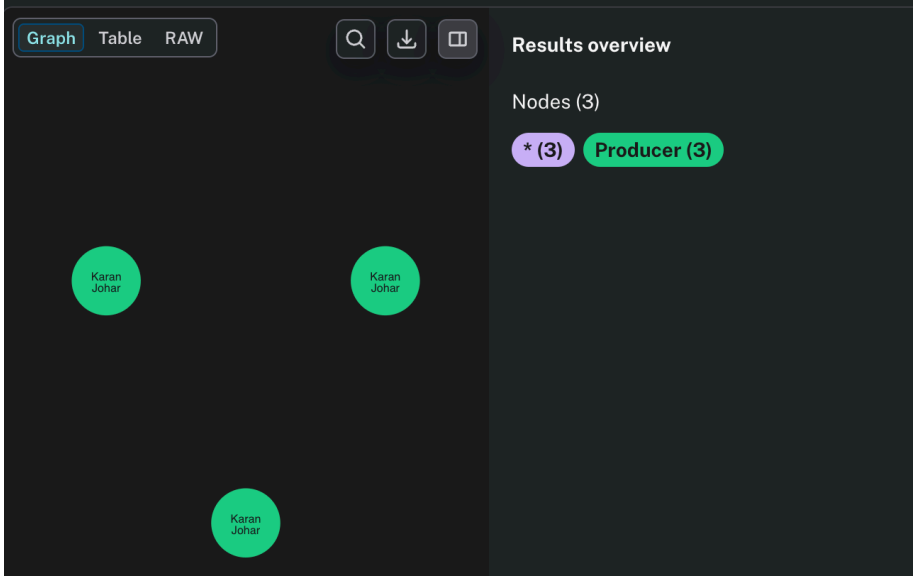
```
neo4j$ CREATE (:Producer {name:'Karan Johar', age:52, city:'Mumbai'
```

Created 1 node, set 3 properties, added 1 label

Completed after 8 ms

2.Displaying created node

```
neo4j$ MATCH (p:Producer {name:'Karan Johar'}) RETURN p;
```



3.Creating a another class of node

```
neo4j$ CREATE (:Movie {name:'DDLJ', year:1995, genre:'Romance'});
```

Created 1 node, set 3 properties, added 1 label

Completed after 154 ms

4.Adding Multiple Nodes

```
CREATE
(:Movie {name:'My Name is Khan', year:2010, genre:'Drama'}),
(:Movie {name:'Kabhi Khushi Kabhie Gham', year:2001,
genre:'Family'}),
(:Producer {name:'Aditya Chopra', age:53, city:'Mumbai'});
```

Created 3 nodes, set 9 properties, added 3 labels

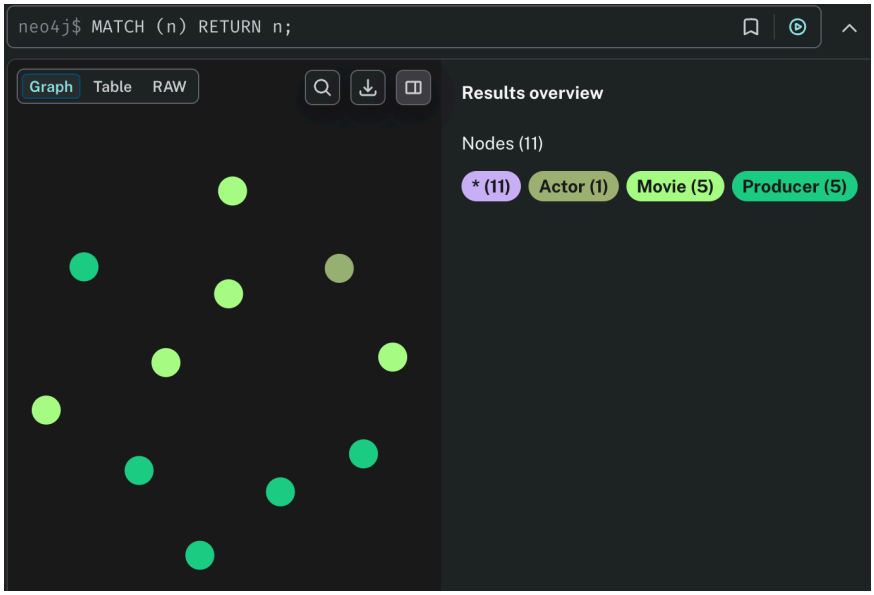
5.Adding ActorNode

```
neo4j$ CREATE (:Actor {name:'Shah Rukh Khan', age:58});
```

Created 1 node, set 2 properties, added 1 label

Comple

6.Displaying all nodes



7.Creating relation between nodes

```
MATCH (p:Producer {name:'Karan Johar'}), (m:Movie {name:'DDLJ'})
CREATE (p)-[:PRODUCED]->(m);
```

Created 3 relationships

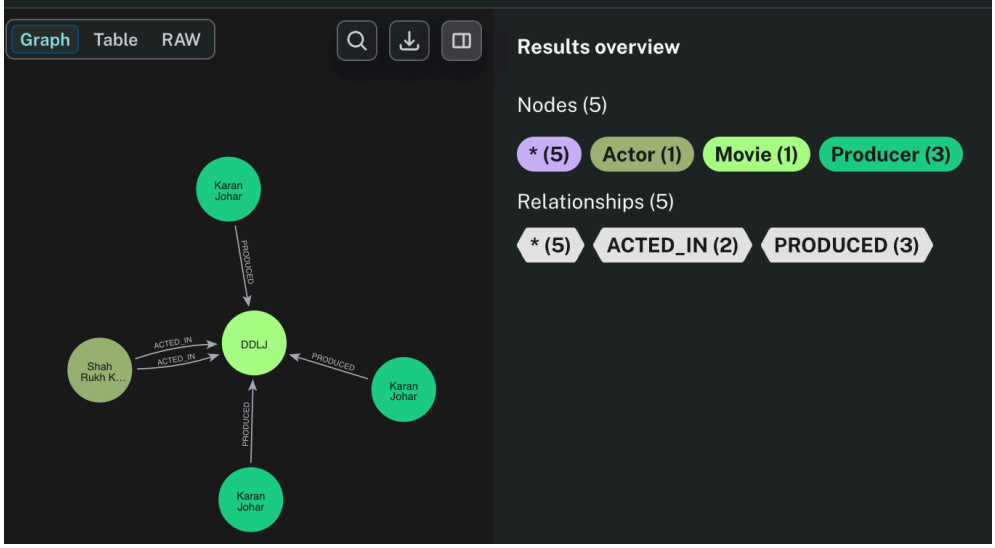
8.Creating relation between multiple nodes

```
MATCH (a:Actor {name:'Shah Rukh Khan'}), (m:Movie {name:'DDLJ'})
CREATE (a)-[:ACTED_IN]->(m);
```

Created 1 relationship

9.Displaying all nodes after relationships

```
neo4j$ MATCH (n)-[r]->(m) RETURN n,r,m;
```



10.Creating index on producer

```
CREATE INDEX producer_index IF NOT EXISTS
FOR (p:Producer) ON (p.name);
```

Added 1 index

11.Creating and displaying movie index

```
CREATE INDEX movie_index IF NOT EXISTS
FOR (m:Movie) ON (m.name);
```

Added 1 index

Complet

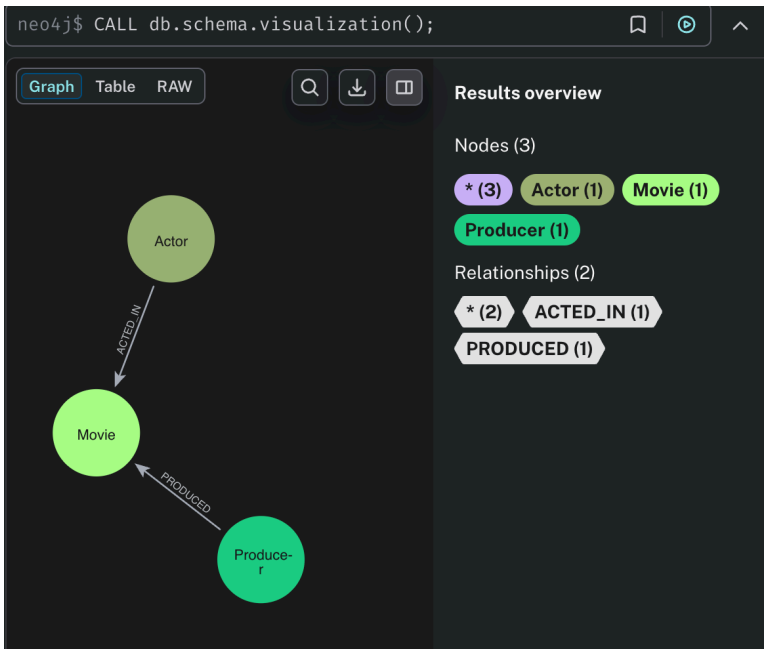
12.Show indexes

```
neo4j$ SHOW INDEXES;
```

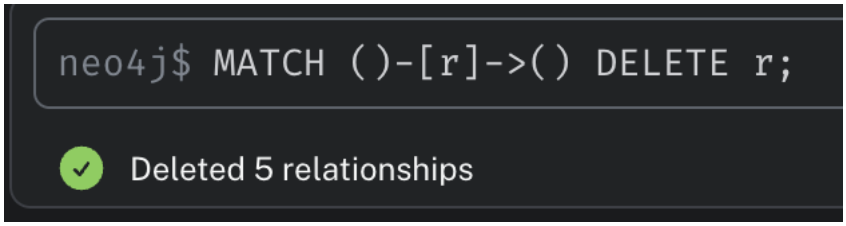
Table RAW

	id	name	state	popul	type	entity	labels	prope	index	ownin	lastR	readC
1	2	"index_1b9dcc97"	"ONLINE"	100.0	"LOOKUP"	"RELATIONSHIP"	null	null	"token-lookup-1.0"	null	null	0
2	1	"index_460996c0"	"ONLINE"	100.0	"LOOKUP"	"NODE"	null	null	"token-lookup-1.0"	null	2026-04-04T17:30:05.844000000Z	14
3	4	"movie_index"	"ONLINE"	100.0	"RANGE"	"NODE"	["Movie"]	["name"]	"range-1.0"	null	null	0
4	3	"producer_index"	"ONLINE"	100.0	"RANGE"	"NODE"	["Producer"]	["name"]	"range-1.0"	null	null	0

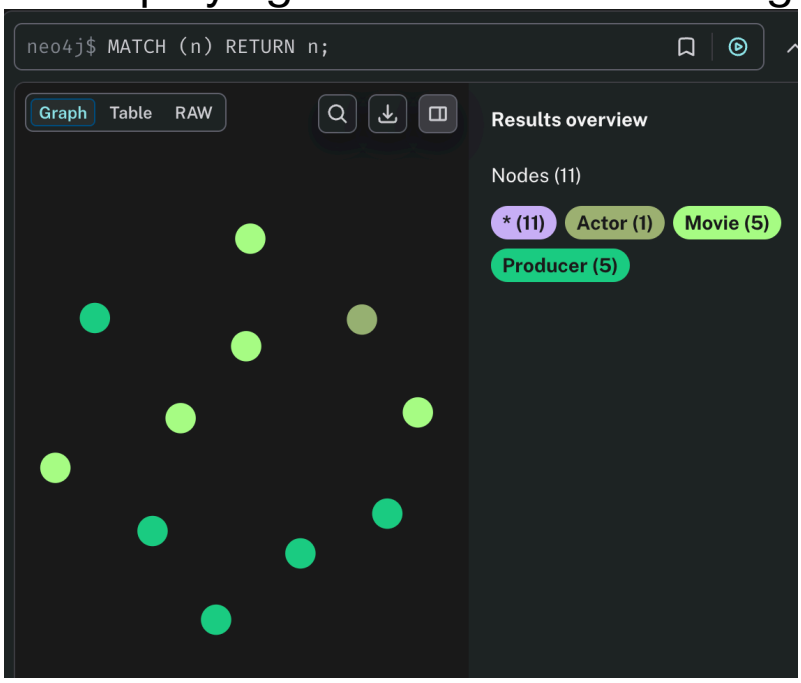
13.Visualising the schema



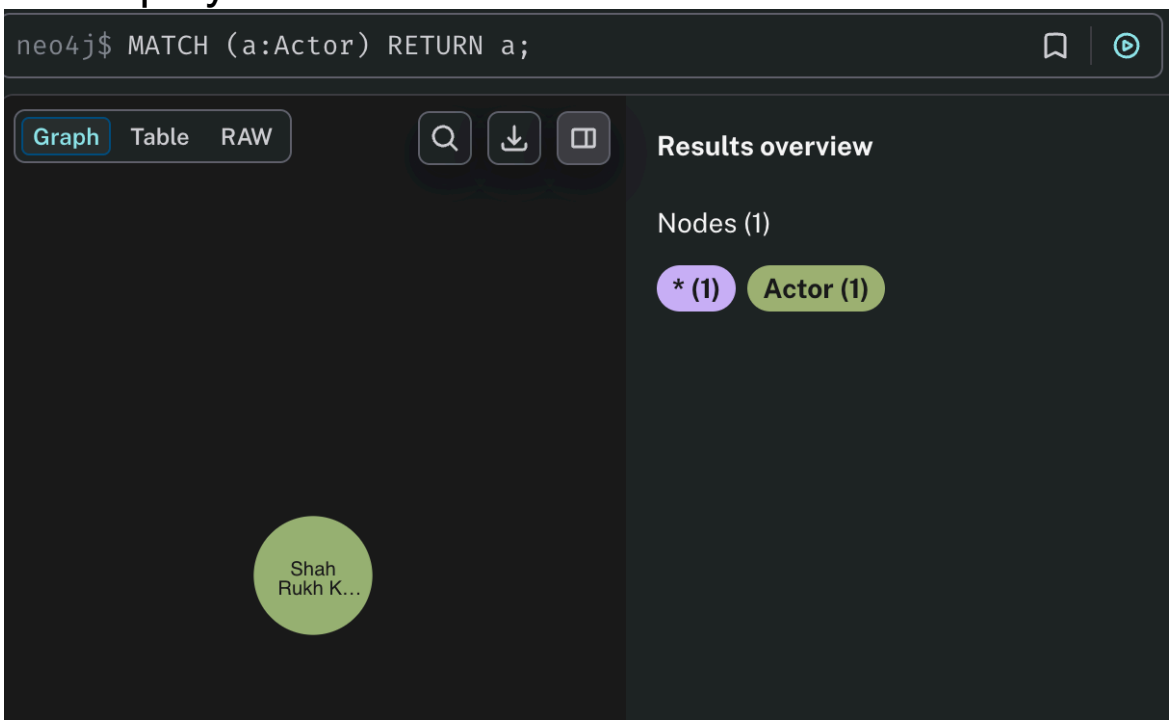
14.Deleting all relationships in the graph



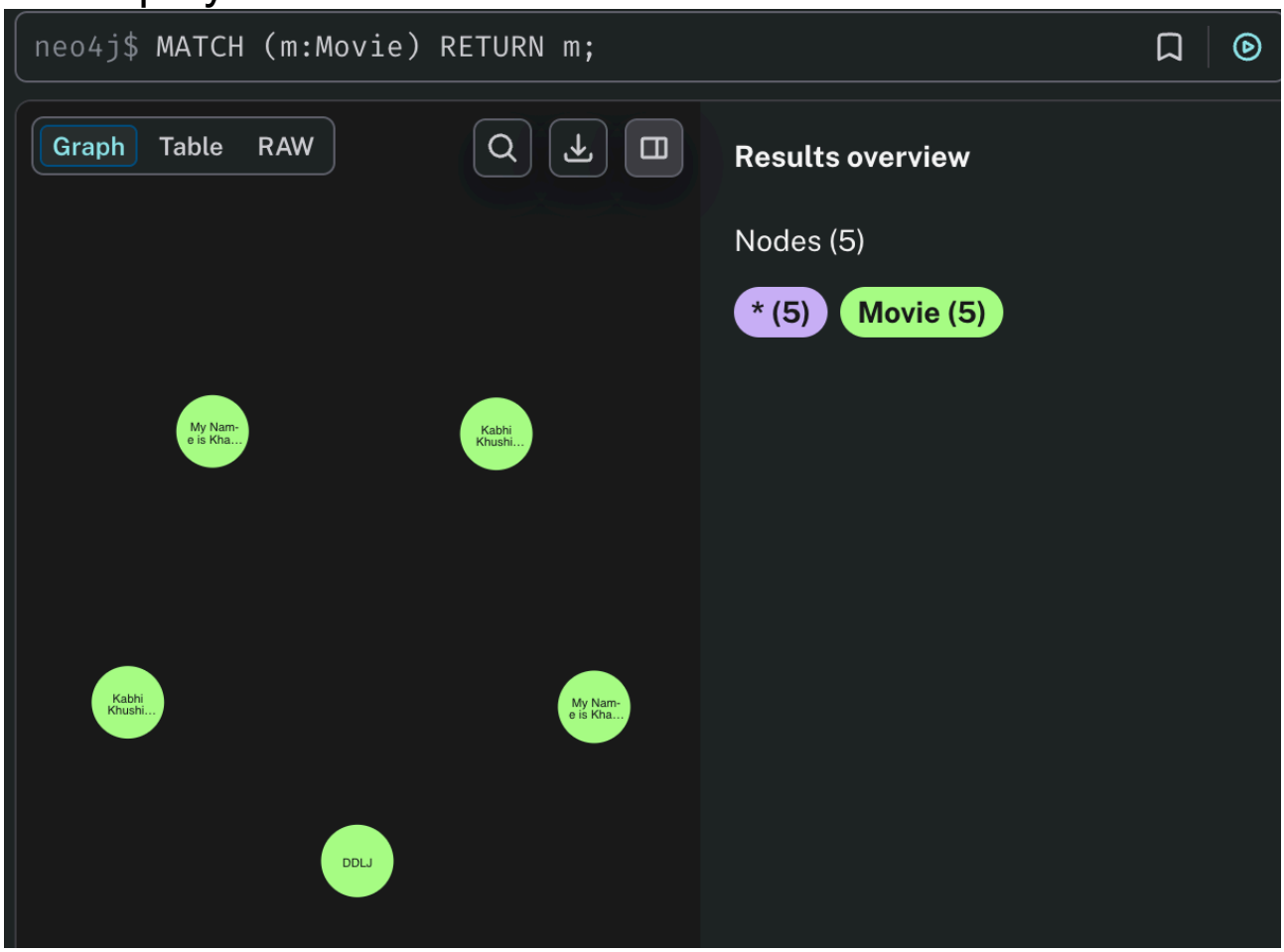
15.Displaying all nodes after deleting relationships



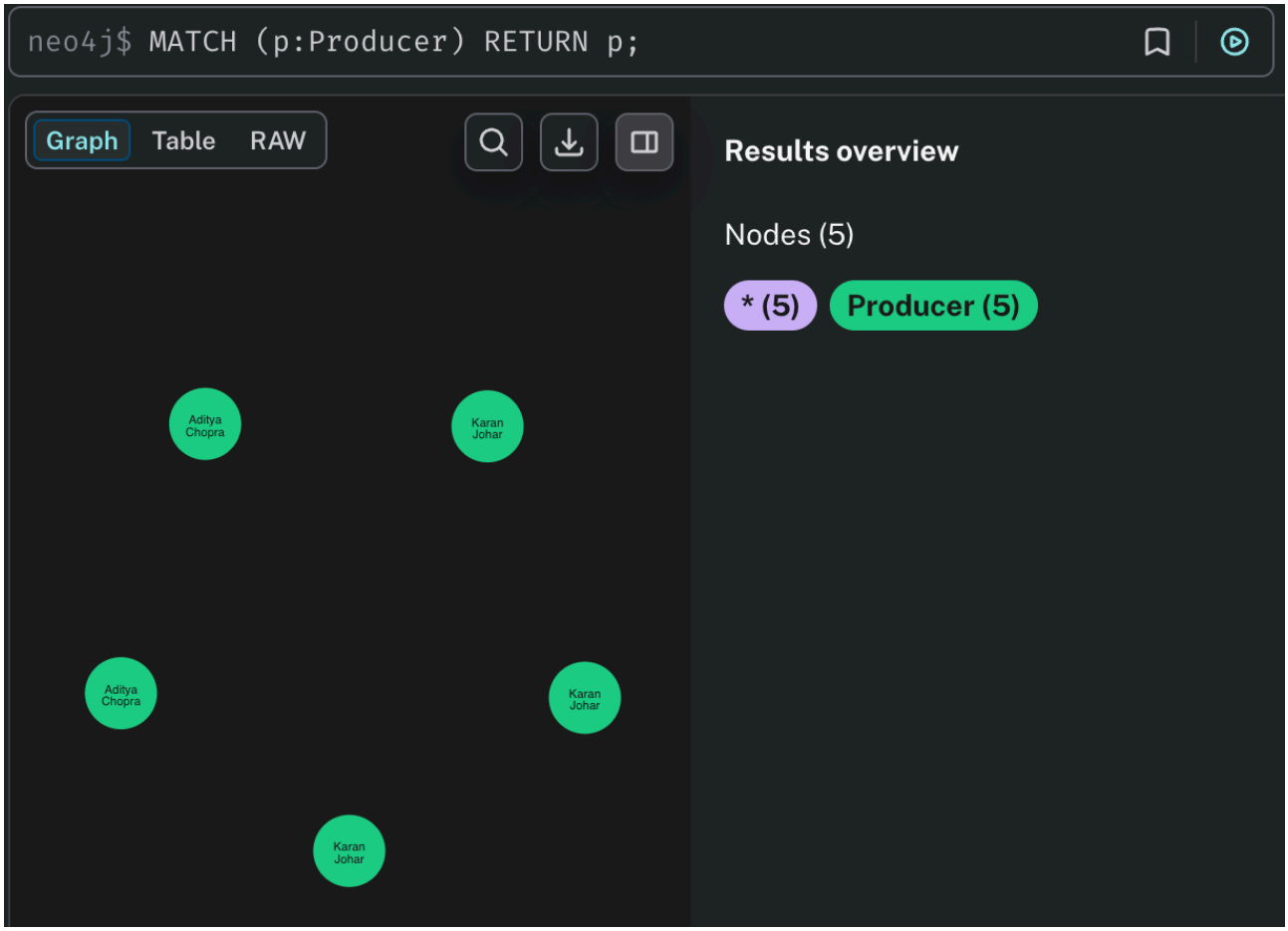
16.Display all Actor nodes



17.Display all Movie nodes



18.Display all Producer nodes



19.Display only DDLJ node

```
neo4j$ MATCH (m:Movie {name:'DDLJ'}) RETURN m;
```

Graph Table RAW

Results overview

Nodes (1)

* (1) Movie (1)

20.Count all nodes

```
neo4j$ MATCH (n) RETURN count(n) AS TotalNodes;
```

Table RAW

TotalNodes
11

21.Count all Actor nodes

```
neo4j$ MATCH (a:Actor) RETURN count(a) AS TotalActors;
```

Table RAW

TotalActors
1

22.Count all Movie nodes

```
neo4j$ MATCH (m:Movie) RETURN count(m) AS TotalMovies;
```

Table RAW

TotalMovies
5

23.Count all Producer nodes

```
neo4j$ MATCH (p:Producer) RETURN count(p) AS TotalProducers;
```

Table RAW

TotalProducers
5

24.Return node names

```
neo4j$ MATCH (n) RETURN labels(n), n.name;
```

Table RAW

labels(n)	n.name
["Producer"]	"Karan Johar"
["Producer"]	"Karan Johar"
["Producer"]	"Karan Johar"
["Movie"]	"DDLJ"
["Actor"]	"Shah Rukh Khan"
["Movie"]	"My Name is Khan"
["Movie"]	"Kabhi Khushi Kabhie Gham"
["Producer"]	"Aditya Chopra"
["Movie"]	"My Name is Khan"
["Movie"]	"Kabhi Khushi Kabhie Gham"

25.Return Actor names

```
neo4j$ MATCH (a:Actor) RETURN a.name;
```

Table RAW

	a.name
1	"Shah Rukh Khan"

26.Return Movie names

```
neo4j$ MATCH (m:Movie) RETURN m.name, m.year, m.genre;
```

Table RAW

	m.name	m.year	m.genre
1	"DDLJ"	1995	"Romance"
2	"My Name is Khan"	2010	"Drama"
3	"Kabhi Khushi Kabhie Gham"	2001	"Family"
4	"My Name is Khan"	2010	"Drama"
5	"Kabhi Khushi Kabhie Gham"	2001	"Family"

27.Return Producer names

```
neo4j$ MATCH (p:Producer) RETURN p.name, p.city;
```

Table RAW

	p.name	p.city
1	"Karan Johar"	"Mumbai"
2	"Karan Johar"	"Mumbai"
3	"Karan Johar"	"Mumbai"
4	"Aditya Chopra"	"Mumbai"
5	"Aditya Chopra"	"Mumbai"

28.Use WHERE with Actor

```
neo4j$ MATCH (a:Actor) WHERE a.age > 50 RETURN a;
```

Graph Table RAW

Results overview

Nodes (1)

* (1) Actor (1)

29.Use WHERE with Movie

```
neo4j$ MATCH (m:Movie) WHERE m.year > 2000 RETURN m;
```

Graph Table RAW

Results overview

Nodes (4)

* (4) **Movie (4)**

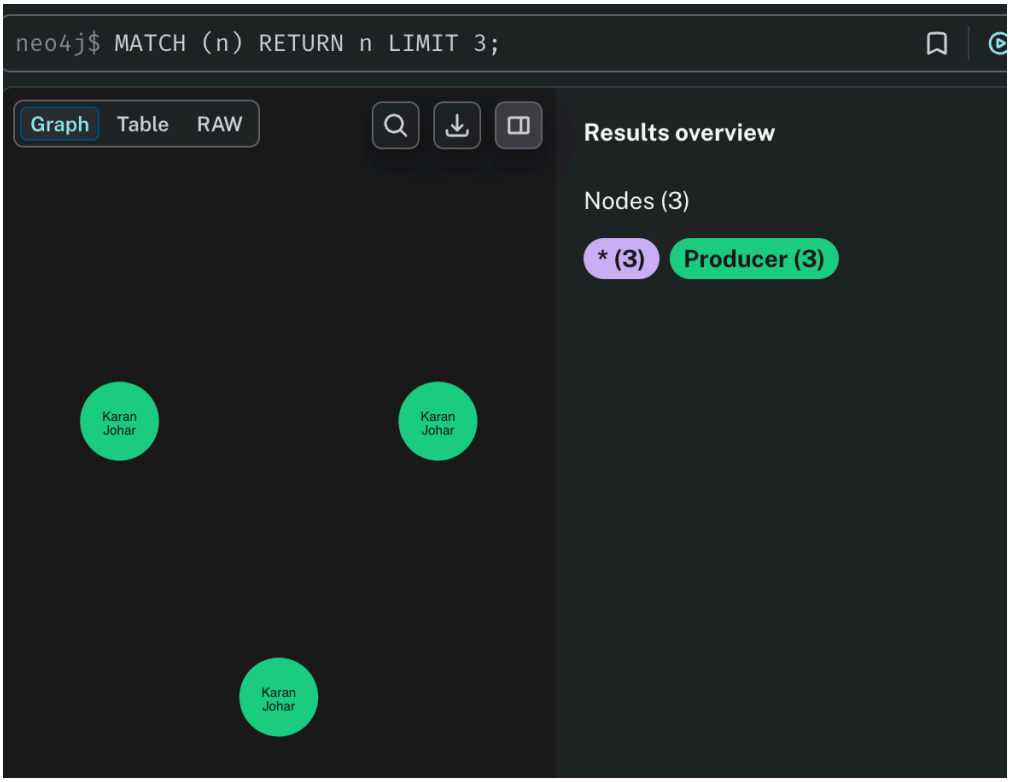
30.Use ORDER BY on movies

```
MATCH (m:Movie)
RETURN m.name, m.year
ORDER BY m.year DESC;
```

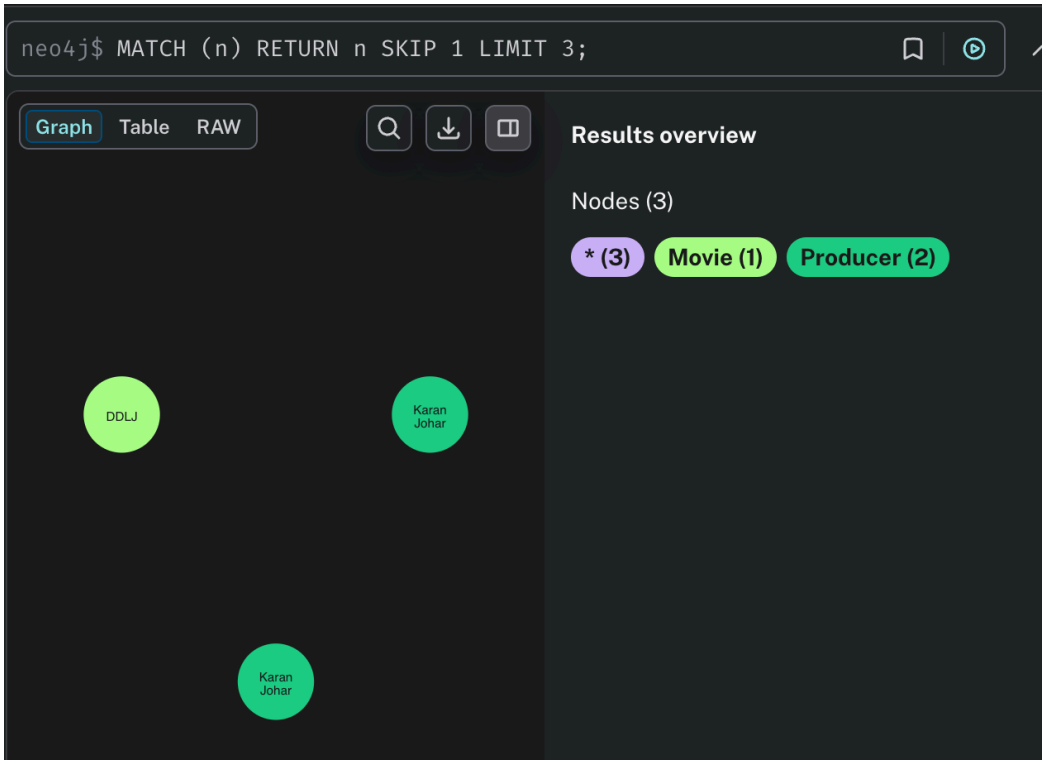
Table RAW

	m.name	m.year
1	"My Name is Khan"	2010
2	"My Name is Khan"	2010
3	"Kabhi Khushi Kabhie Gham"	2001
4	"Kabhi Khushi Kabhie Gham"	2001
5	"DDLJ"	1995

31.Use LIMIT



32.Use SKIP



33.Merge a new node

```
MERGE (n:Name1 {name:'Meena'})
RETURN n;
```

Graph Table RAW

Results overview

Nodes (1)

* (1) Name1 (1)

Meena

34.Merge another new node

```
MERGE (n:Name3 {name:'Ratna'})
RETURN n;
```

Graph Table RAW

Results overview

Nodes (1)

* (1) Name3 (1)

Ratna

35.Display system info

```
neo4j$ CALL dbms.components();
```

Table RAW

	name	versions	edition
1	"Neo4j Kernel"	["2026.03.1"]	"community"
2	"Cypher"	["5", "25"]	""

36.Displaying distinct labels in the graph

```
neo4j$ MATCH (n) RETURN DISTINCT labels(n);
```

Table RAW

	labels(n)
1	["Producer"]
2	["Movie"]
3	["Actor"]
4	["Name1"]
5	["Name3"]

37.Show Actor names with ages

```
neo4j$ MATCH (a:Actor) RETURN a.name, a.age;
```

Table RAW

	a.name	a.age
1	"Shah Rukh Khan"	58

38.Show movies released before 2005

```
neo4j$ MATCH (m:Movie) WHERE m.year < 2005 RETURN m.name, m.yea
```

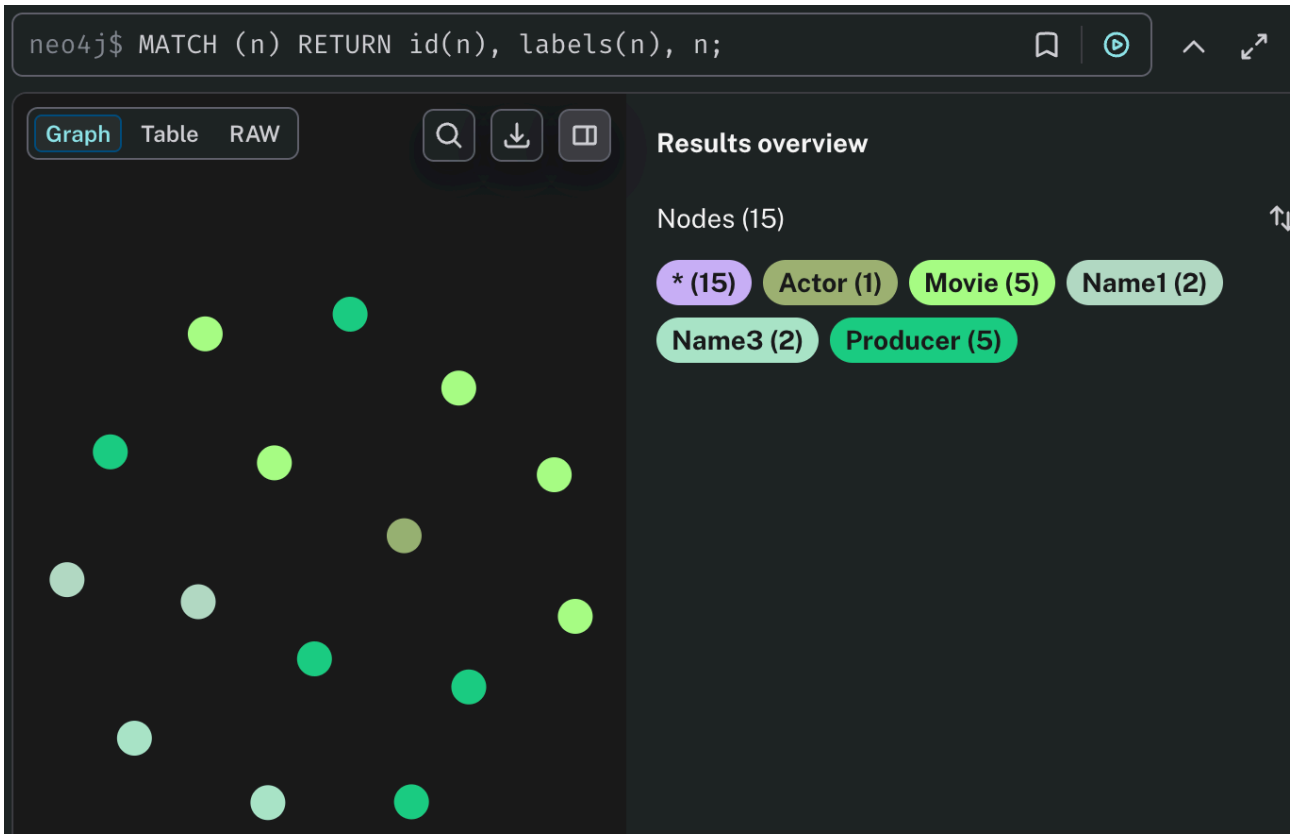
Table RAW

	m.name	m.year
1	"DDLJ"	1995
2	"Kabhi Khushi Kabhie Gham"	2001
3	"Kabhi Khushi Kabhie Gham"	2001

39.Sort producer names in descending order

```
neo4j$ MATCH (p:Producer) RETURN p.name ORDER BY p.name DESC;
```

Table RAW

	p.name
1	"Karan Johar"
2	"Karan Johar"
3	"Karan Johar"
4	"Aditya Chopra"
5	"Aditya Chopra"

40.Displaying node IDs, labels, and properties